

Duplicate Question Pairs Detection Model

Technical Documentation

Priyansh Singh

Contents

1	Project Overview	3
1.1	Problem Statement	3
1.2	Approach	3
2	Text Vectorization Techniques	3
2.1	Bag of Words (BoW)	3
2.1.1	Concept	3
2.1.2	Numerical Example	3
2.2	TF-IDF (Term Frequency - Inverse Document Frequency)	4
2.2.1	Concept	4
2.2.2	Numerical Example	4
2.3	Word2Vec	5
2.3.1	Concept	5
2.3.2	Skip-gram Example	5
2.3.3	Mathematical Representation	5
2.3.4	Numerical Example (Simplified)	6
2.3.5	Document Representation	6
3	Feature Engineering	7
3.1	Custom Features	7
3.1.1	1. Common Words Count	7
3.1.2	2. Total Words	7
3.1.3	3. Length Features	7
3.1.4	4. Token Features	8
3.1.5	5. Fuzziness Features	8
4	Complete Feature Vector	9
5	Machine Learning Models	9
5.1	Random Forest	9
5.1.1	How it Works	9
5.1.2	Example Decision Tree Split	9
5.1.3	Why Random Forest Works Well	10
5.2	XGBoost (Extreme Gradient Boosting)	10

5.2.1	How it Works	10
5.2.2	Objective Function	10
5.2.3	Numerical Example	10
5.2.4	Why XGBoost Works Better	11
6	Model Performance	11
6.1	Results	11
6.2	Why Word2Vec Performed Best	11
6.3	Evaluation Metrics	12
7	Interview Defense Points	12
7.1	Key Points to Emphasize	12
7.2	Common Interview Questions & Answers	13
7.2.1	Q1: Why use three different vectorization methods?	13
7.2.2	Q2: Why did Word2Vec outperform others?	13
7.2.3	Q3: Explain fuzziness features	13
7.2.4	Q4: Why XGBoost over Random Forest?	13
7.2.5	Q5: How would you improve the model further?	14
7.2.6	Q6: How do you handle class imbalance?	14
7.3	Technical Depth Questions	14
7.3.1	Q: What's the complexity of computing cosine similarity?	14

1 Project Overview

1.1 Problem Statement

The goal is to identify whether two questions posted on forum pages are duplicates (asking the same thing) or not. This helps in:

- Reducing redundant content on forum pages
- Directing users to existing answers
- Improving search and recommendation systems
- Enhancing user experience across organizational forums

1.2 Approach

The solution combines three types of text vectorization techniques (Bag of Words, TF-IDF, Word2Vec) with custom feature engineering to train machine learning models (Random Forest and XGBoost).

2 Text Vectorization Techniques

2.1 Bag of Words (BoW)

2.1.1 Concept

Bag of Words represents text as a collection of words, ignoring grammar and word order but keeping track of word frequency.

2.1.2 Numerical Example

Consider two questions:

- **Q1:** “How do I learn Python programming?”
- **Q2:** “What is the best way to learn Python?”

Step 1: Create Vocabulary

After preprocessing (lowercase, remove stopwords):

Q1: [“learn”, “python”, “programming”]

Q2: [“best”, “way”, “learn”, “python”]

Vocabulary: $V = \{\text{best, learn, programming, python, way}\}$

Step 2: Create Vectors

$$Q1 = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \quad Q2 = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 1 \end{bmatrix} \tag{1}$$

where the dimensions correspond to: [best, learn, programming, python, way]

Step 3: Calculate Similarity

Using cosine similarity:

$$\text{cosine_similarity}(Q1, Q2) = \frac{Q1 \cdot Q2}{\|Q1\| \times \|Q2\|} = \frac{2}{\sqrt{3} \times \sqrt{4}} = \frac{2}{\sqrt{12}} \approx 0.577 \quad (2)$$

2.2 TF-IDF (Term Frequency - Inverse Document Frequency)

2.2.1 Concept

TF-IDF weighs words by their importance. Common words across all documents get lower weights, while unique words get higher weights.

Formula:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t) \quad (3)$$

where:

$$\text{TF}(t, d) = \frac{\text{frequency of term } t \text{ in document } d}{\text{total terms in document } d} \quad (4)$$

$$\text{IDF}(t) = \log \left(\frac{\text{total number of documents}}{\text{number of documents containing term } t} \right) \quad (5)$$

2.2.2 Numerical Example

Consider a corpus of 3 questions:

- **D1:** “learn python programming”
- **D2:** “learn python best way”
- **D3:** “java programming language”

Calculate TF for word “python” in D1:

$$\text{TF}(\text{python}, D1) = \frac{1}{3} = 0.333 \quad (6)$$

Calculate IDF for word “python”:

$$\text{IDF}(\text{python}) = \log \left(\frac{3}{2} \right) = \log(1.5) \approx 0.176 \quad (7)$$

(“python” appears in D1 and D2, so 2 documents)

Calculate TF-IDF:

$$\text{TF-IDF}(\text{python}, D1) = 0.333 \times 0.176 \approx 0.059 \quad (8)$$

For word “programming” in D1:

$$\text{TF}(\text{programming}, D1) = \frac{1}{3} = 0.333 \quad (9)$$

$$\text{IDF}(\text{programming}) = \log \left(\frac{3}{2} \right) \approx 0.176 \quad (10)$$

$$\text{TF-IDF}(\text{programming}, D1) = 0.333 \times 0.176 \approx 0.059 \quad (11)$$

Complete TF-IDF Vector for D1:

Term	TF	IDF	TF-IDF
learn	0.333	0.176	0.059
python	0.333	0.176	0.059
programming	0.333	0.176	0.059

Table 1: TF-IDF calculation for Document D1

2.3 Word2Vec

2.3.1 Concept

Word2Vec creates dense vector representations where semantically similar words have similar vectors. Unlike BoW and TF-IDF (sparse vectors), Word2Vec produces dense vectors (typically 100-300 dimensions).

Two architectures:

- **CBOW (Continuous Bag of Words):** Predicts target word from context words
- **Skip-gram:** Predicts context words from target word

2.3.2 Skip-gram Example

Training sentence: “The cat sat on the mat”

Window size = 2

For target word “sat”, the context pairs would be:

Input → Output
 sat → the
 sat → cat
 sat → on
 sat → the

2.3.3 Mathematical Representation

For a vocabulary of size V and embedding dimension d :

- Input word: one-hot encoded vector $\mathbf{x} \in \mathbb{R}^V$
- Weight matrix: $\mathbf{W} \in \mathbb{R}^{V \times d}$
- Context matrix: $\mathbf{W}' \in \mathbb{R}^{d \times V}$

Forward pass:

$$\mathbf{h} = \mathbf{W}^T \mathbf{x} \quad (\text{hidden layer, embedding}) \quad (12)$$

$$\mathbf{u} = \mathbf{W}'^T \mathbf{h} \quad (\text{output scores}) \quad (13)$$

$$\mathbf{y} = \text{softmax}(\mathbf{u}) \quad (\text{output probabilities}) \quad (14)$$

where:

$$\text{softmax}(u_j) = \frac{\exp(u_j)}{\sum_{j'=1}^V \exp(u_{j'})} \quad (15)$$

2.3.4 Numerical Example (Simplified)

Let's use a tiny vocabulary: $V = \{\text{cat}, \text{sat}, \text{mat}\}$ with $d = 2$ dimensions.

Step 1: Initialize random weights

$$\mathbf{W} = \begin{bmatrix} 0.1 & 0.3 \\ 0.2 & 0.5 \\ 0.4 & 0.1 \end{bmatrix}, \quad \mathbf{W}' = \begin{bmatrix} 0.2 & 0.4 & 0.3 \\ 0.1 & 0.3 & 0.5 \end{bmatrix} \quad (16)$$

Step 2: Input word “sat” (one-hot encoded)

$$\mathbf{x}_{\text{sat}} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \quad (17)$$

Step 3: Get embedding

$$\mathbf{h} = \mathbf{W}^T \mathbf{x}_{\text{sat}} = \begin{bmatrix} 0.1 & 0.2 & 0.4 \\ 0.3 & 0.5 & 0.1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.2 \\ 0.5 \end{bmatrix} \quad (18)$$

This $[0.2, 0.5]$ is the Word2Vec embedding for “sat”!

Step 4: Predict context word “cat”

$$\mathbf{u} = \mathbf{W}'^T \mathbf{h} = \begin{bmatrix} 0.2 & 0.1 \\ 0.4 & 0.3 \\ 0.3 & 0.5 \end{bmatrix} \begin{bmatrix} 0.2 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 0.09 \\ 0.23 \\ 0.31 \end{bmatrix} \quad (19)$$

Step 5: Apply softmax

$$P(\text{cat}|\text{sat}) = \frac{\exp(0.09)}{\exp(0.09) + \exp(0.23) + \exp(0.31)} \approx 0.305 \quad (20)$$

$$P(\text{sat}|\text{sat}) = \frac{\exp(0.23)}{\exp(0.09) + \exp(0.23) + \exp(0.31)} \approx 0.351 \quad (21)$$

$$P(\text{mat}|\text{sat}) = \frac{\exp(0.31)}{\exp(0.09) + \exp(0.23) + \exp(0.31)} \approx 0.344 \quad (22)$$

2.3.5 Document Representation

For a question with multiple words, average the word vectors:

$$\mathbf{v}_{\text{document}} = \frac{1}{n} \sum_{i=1}^n \mathbf{v}_{\text{word}_i} \quad (23)$$

Example: “learn python programming”

$$\mathbf{v}_{\text{learn}} = [0.2, 0.5, -0.3, \dots] \quad (24)$$

$$\mathbf{v}_{\text{python}} = [0.1, 0.6, -0.2, \dots] \quad (25)$$

$$\mathbf{v}_{\text{programming}} = [0.3, 0.4, -0.1, \dots] \quad (26)$$

$$\mathbf{v}_{\text{question}} = \frac{1}{3}([0.2, 0.5, -0.3] + [0.1, 0.6, -0.2] + [0.3, 0.4, -0.1]) \quad (27)$$

$$= [0.2, 0.5, -0.2, \dots] \quad (28)$$

3 Feature Engineering

3.1 Custom Features

Beyond vector representations, additional features capture linguistic similarities:

3.1.1 1. Common Words Count

Definition: Number of words that appear in both questions.

Example:

Q1: “How to learn Python programming?”

Q2: “What is the best way to learn Python?”

After preprocessing:

Q1 = {learn, python, programming}

Q2 = {best, way, learn, python}

Common words = {learn, python}

common_words_count = 2

3.1.2 2. Total Words

Features:

- total_words_Q1: Number of words in question 1
- total_words_Q2: Number of words in question 2
- total_words_both: Sum of words in both questions

Example:

total_words_Q1 = 3 (29)

total_words_Q2 = 4 (30)

total_words_both = 7 (31)

3.1.3 3. Length Features

Character-level features:

- len_Q1: Character length of question 1
- len_Q2: Character length of question 2
- len_diff: Absolute difference in lengths
- len_ratio: Ratio of lengths

Example:

len_Q1 = 32 characters (32)

len_Q2 = 41 characters (33)

len_diff = $|32 - 41| = 9$ (34)

len_ratio = $\frac{\min(32, 41)}{\max(32, 41)} = \frac{32}{41} \approx 0.78$ (35)

3.1.4 4. Token Features

Tokens can include words, punctuation, special characters:

- token_count_Q1
- token_count_Q2
- common_tokens

3.1.5 5. Fuzziness Features

Fuzziness measures string similarity using edit distance algorithms.

a) Fuzzy Ratio:

Uses Levenshtein distance (minimum number of single-character edits).

$$\text{fuzzy_ratio} = \frac{\text{matching characters}}{\text{total characters}} \times 100 \quad (36)$$

Example:

Q1: “learn python”

Q2: “learn java”

The library calculates similarity:

$$\text{fuzzy_ratio}(Q1, Q2) = 67\% \quad (37)$$

b) Partial Ratio:

Finds best matching substring.

Example:

Q1: “How to learn Python programming”

Q2: “learn Python”

$$\text{partial_ratio}(Q1, Q2) = 100\% \quad (38)$$

(“learn Python” is a perfect substring match)

c) Token Sort Ratio:

Sorts tokens alphabetically before comparing.

Example:

Q1: “python learn programming”

Q2: “learn programming python”

After sorting both become: “learn programming python”

$$\text{token_sort_ratio}(Q1, Q2) = 100\% \quad (39)$$

d) Token Set Ratio:

Uses set intersection and differences.

$$\text{token_set_ratio} = \max \left(\frac{|A \cap B|}{|A|}, \frac{|A \cap B|}{|B|}, \frac{|A \cap B|}{|A \cup B|} \right) \times 100 \quad (40)$$

4 Complete Feature Vector

For a question pair, the final feature vector combines:

$$\mathbf{X} = [\mathbf{v}_{\text{BoW}}, \mathbf{v}_{\text{TF-IDF}}, \mathbf{v}_{\text{W2V}}, \mathbf{f}_{\text{custom}}] \quad (41)$$

where $\mathbf{f}_{\text{custom}}$ includes:

$$\mathbf{f}_{\text{custom}} = [\text{common_words}, \text{total_words_Q1}, \text{total_words_Q2}, \\ \text{len_Q1}, \text{len_Q2}, \text{len_diff}, \text{len_ratio}, \\ \text{token_count_Q1}, \text{token_count_Q2}, \\ \text{fuzzy_ratio}, \text{partial_ratio}, \text{token_sort_ratio}, \\ \text{token_set_ratio}] \quad (42)$$

5 Machine Learning Models

5.1 Random Forest

5.1.1 How it Works

Random Forest is an ensemble of decision trees:

1. Create N decision trees (e.g., 100 trees)
2. Each tree is trained on a random subset of data (bootstrap sample)
3. Each split considers a random subset of features
4. For prediction: each tree votes, majority wins

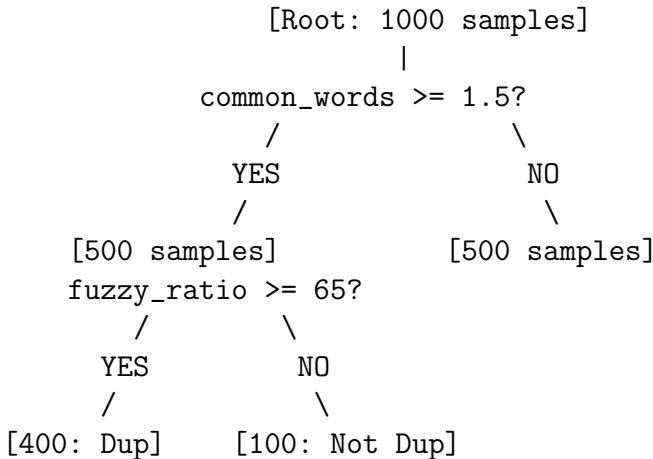
Prediction:

$$\hat{y} = \text{mode}\{h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_N(\mathbf{x})\} \quad (43)$$

where h_i is the i -th decision tree.

5.1.2 Example Decision Tree Split

Consider features: [common_words=2, fuzzy_ratio=75, len_diff=10]



5.1.3 Why Random Forest Works Well

- Handles high-dimensional data (many features)
- Captures non-linear relationships
- Reduces overfitting through averaging
- Provides feature importance

5.2 XGBoost (Extreme Gradient Boosting)

5.2.1 How it Works

XGBoost builds trees sequentially, where each tree corrects errors of previous trees.

Algorithm:

1. Start with initial prediction: $\hat{y}^{(0)} = 0.5$
2. For each iteration $t = 1, 2, \dots, T$:
 - Calculate residuals: $r_i = y_i - \hat{y}_i^{(t-1)}$
 - Train tree f_t to predict residuals
 - Update: $\hat{y}^{(t)} = \hat{y}^{(t-1)} + \eta \cdot f_t(\mathbf{x})$

where η is the learning rate.

5.2.2 Objective Function

$$\text{Obj}^{(t)} = \sum_{i=1}^n L(y_i, \hat{y}_i^{(t)}) + \sum_{k=1}^t \Omega(f_k) \quad (44)$$

where:

- L is the loss function (e.g., log loss for classification)
- $\Omega(f_k)$ is regularization term

Regularization:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (45)$$

where T is number of leaves, w_j is leaf weight.

5.2.3 Numerical Example

Dataset:

Iteration 1:

$$\begin{aligned} \text{Residuals: } r_1 &= 1 - 0.5 = 0.5 \\ r_2 &= 0 - 0.5 = -0.5 \\ r_3 &= 1 - 0.5 = 0.5 \end{aligned} \quad (46)$$

Feature	True Label	Prediction
x=0.8	1 (Duplicate)	0.5
x=0.3	0 (Not Duplicate)	0.5
x=0.9	1	0.5

Train tree to predict residuals with $\eta = 0.1$:

$$\hat{y}_1^{(1)} = 0.5 + 0.1 \times 0.5 = 0.55 \quad (47)$$

$$\hat{y}_2^{(1)} = 0.5 + 0.1 \times (-0.5) = 0.45 \quad (48)$$

$$\hat{y}_3^{(1)} = 0.5 + 0.1 \times 0.5 = 0.55 \quad (49)$$

Continue for more iterations...

5.2.4 Why XGBoost Works Better

- Gradient-based optimization
- Built-in regularization prevents overfitting
- Handles missing values automatically
- Parallel processing for speed
- Often achieves state-of-the-art results

6 Model Performance

6.1 Results

Model	Vectorization	Accuracy	Features
Random Forest	Word2Vec	80.1%	300 (W2V) + 13 (custom)
XGBoost	Word2Vec	80.64%	300 (W2V) + 13 (custom)

Table 2: Model performance comparison

6.2 Why Word2Vec Performed Best

1. **Semantic understanding:** Captures meaning, not just word matching
2. **Dense representations:** More efficient than sparse BoW/TF-IDF
3. **Handles synonyms:** “learn” and “study” have similar vectors
4. **Context awareness:** Word meaning depends on surrounding words

Example:

Q1: “How to study machine learning?”

Q2: “Best way to learn ML?”

- BoW/TF-IDF: Low similarity (different words)
- Word2Vec: High similarity (“study” $\approx$ “learn”, “machine learning” $\approx$ “ML”)

6.3 Evaluation Metrics

Accuracy:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \quad (50)$$

Additional metrics you should know:

Precision:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (51)$$

Recall:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (52)$$

F1-Score:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (53)$$

where:

- TP = True Positives (correctly identified duplicates)
- FP = False Positives (incorrectly identified as duplicates)
- FN = False Negatives (missed duplicates)

7 Interview Defense Points

7.1 Key Points to Emphasize

1. **Multi-faceted approach:** Combined multiple vectorization techniques
2. **Feature engineering:** Created domain-specific features beyond basic text representation
3. **Model selection:** Tested multiple algorithms, chose best performers
4. **Strong results:** 80%+ accuracy is solid for text similarity tasks

7.2 Common Interview Questions & Answers

7.2.1 Q1: Why use three different vectorization methods?

Answer: Each captures different aspects:

- BoW: Basic word presence
- TF-IDF: Word importance
- Word2Vec: Semantic meaning

Combining them gives a richer representation. We experimented and found Word2Vec with custom features performed best.

7.2.2 Q2: Why did Word2Vec outperform others?

Answer: Word2Vec captures semantic similarity. For example, “How to learn Python?” and “Best way to study Python?” use different words (learn vs study) but have similar meaning. BoW/TF-IDF would show low similarity, but Word2Vec recognizes “learn” and “study” are semantically close.

7.2.3 Q3: Explain fuzziness features

Answer: Fuzziness uses string matching algorithms:

- Fuzzy ratio: Overall string similarity using edit distance
- Partial ratio: Finds best substring match
- Token sort ratio: Handles word order differences
- Token set ratio: Uses set operations for comparison

These capture surface-level similarity that complements semantic embeddings.

7.2.4 Q4: Why XGBoost over Random Forest?

Answer: XGBoost typically outperforms Random Forest because:

- Sequential learning corrects previous errors
- Built-in regularization prevents overfitting
- More efficient optimization

Our 80.64% vs 80.1% shows marginal improvement, validating this.

7.2.5 Q5: How would you improve the model further?

Answer:

- Try transformer models (BERT, Sentence-BERT) for better semantic understanding
- Ensemble multiple models
- Add more features: POS tags, named entities, question type
- Use larger Word2Vec embeddings (e.g., pre-trained on Google News)
- Hyperparameter tuning (grid search, random search)

7.2.6 Q6: How do you handle class imbalance?

Answer:

- Use weighted loss functions
- SMOTE for synthetic minority samples
- Adjust class weights in Random Forest/XGBoost
- Evaluate with F1-score instead of just accuracy

7.3 Technical Depth Questions

7.3.1 Q: What's the complexity of computing cosine similarity?

Answer: $O(d)$ where d is the vector dimension. For sparse vectors (BoW), only non-zero elements contribute.